

JEET THAKKAR
Software Developer (Backend | Distributed Systems)
Windsor, ON, Canada
jeetthakkar2612@gmail.com | github.com/jeet7122 | jeet7122.github.io

SUMMARY

Backend engineer with hands-on experience designing and delivering **distributed, event-driven systems** using Java, Spring Boot, and Kafka. Proven track record of improving system performance, scalability, and security in production environments. Strong in building **low-latency APIs, real-time pipelines, and microservices architectures with reliable, idempotent event processing**

SKILLS

Languages: Java, TypeScript, SQL
Backend: Spring Boot, REST APIs, Microservices, Spring Security, JWT/OAuth2.0, RBAC
Data & Messaging: Apache Kafka, Redis, WebSockets, Event-Driven Architecture
Databases: PostgreSQL, MongoDB, Query Optimization, Indexing
DevOps: Docker, AWS S3, CI/CD, Linux, Oracle VM
CS Fundamentals: Distributed Systems, System Design, OOP, Data Structures & Algorithms
Testing: JUnit, Mockito, Integration Testing
Frontend: Next.js, React (basic / full-stack Support)

EXPERIENCE

Software Developer (Contract) - Bridge-City Solutions | 2025 - Present

- **Improved API latency by 35%** by redesigning 12 high-traffic PostgreSQL endpoints (query optimization, connection pooling, caching), reducing response time from **~180ms** → **~115ms**.
 - **Eliminated security vulnerabilities across 6 microservices** by implementing JWT-based authentication and RBAC using Spring Security (replacing legacy session-based auth).
 - Cut manual workflow **processing time by ~60%**: engineered asynchronous, event-driven pipelines using Spring Boot and lightweight internal event bus for low-latency service communication, replacing synchronous blocking calls and enabling real-time system state updates without polling.
 - **Reduced deployment failures by 50%** by containerizing 4 services with Docker and standardizing deployment workflows
 - **Delivered production-ready e-commerce backend in <3 weeks**, owning system design, schema modeling, and API development end-to-end.
-

PROJECTS

1st Place - IT Club St. Clair Hackathon 2026 | Schedule Processing System - Next.js, TypeScript | [Repo](#)

Built under a 4-hour hackathon constraint; placed 1st out of 6 competing teams.

- Processed 100% of heterogeneous schedule datasets without data loss: engineered a type-safe TypeScript ingestion pipeline that validated and normalized 3 structurally different JSON schemas (classes, exams, timetables) — catching null values, malformed timestamps, and schema drift before they reached the UI layer.
- Reduced conflict-related scheduling errors to zero in test datasets: designed and implemented a conflict-detection algorithm that identified overlapping events across 500+ entries in under 200 ms, surfacing actionable warnings directly in the calendar UI.
- **Improved debugging speed by ~40% over a naive approach**: aggregated validation errors into structured, categorized reports rather than failing fast on first error — enabling teammates to resolve all 37 edge-case issues in a single pass rather than iterative re-runs.

Scan Forge - Distributed Code Scanning Platform - Java, Spring Boot, Kafka, WebSockets, Next.js, Docker | [Repo](#) | [Live](#)

- Achieved sub-2-second end-to-end scan feedback for 95% of requests: architected an event-driven microservices system using Apache Kafka with idempotent producers to decouple file ingestion, static analysis, AI enrichment, and result delivery - eliminating synchronous blocking and enabling parallel processing across 4 independent services.
- Scaled file analysis throughput to 10+ concurrent scans without degradation: leveraged Kafka consumer groups to distribute scan workloads horizontally, allowing the scanner service to process multiple files in parallel versus the single-threaded baseline.
- **Reduced UI polling overhead to zero**: implemented real-time scan-progress updates via WebSockets, replacing a client-polling pattern that had generated unnecessary server load on every 2-second interval.
- Achieved zero inter-service contract regressions across 20+ iterations: defined explicit Kafka message schemas as shared contracts between producer and consumer services, preventing the integration errors common in loosely coupled distributed systems.

Cloud Vault - Event-Driven Storage Platform - Spring Boot, Kafka, PostgreSQL, Next.js, Docker, AWS S3 | [Repo](#) | [Live](#)

- **Reduced file processing latency by ~70%**: migrated a monolithic synchronous upload handler to an event-driven architecture using Kafka - decoupling HTTP ingestion from S3 storage and database writes, cutting average processing time from ~3 s to ~900 ms per file.
 - **Improved API performance by 45%** with optimized queries and cursor-based pagination (**p95 < 80ms at 200 concurrent requests**).
 - Enabled **horizontal scalability with zero code changes** using distributed Kafka worker architecture.
-

EDUCATION

Diploma in Computer Programming
St. Clair College | 2024 - 2026
Relevant Coursework: ["Data Structures and Algorithm", "System Design", "OOP", "Networking", "Concurrency"]